

Python 程序设计

第 7 讲 (a): Matplotlib 绘图基础

HY Deng

hydeng@shiep.edu.cn



本讲的主要内容

1 1. 画出第一张“有信息的图”

2 2. 多条曲线、图例

3 3. 图形定制：绘图属性

- 3.1 绘图属性概述

- 3.2 字体大小、线型、颜色与点标记

- 3.3 图形显示中文

Matplotlib 在 Python 中的位置

■ Python 应用的完整流程:

建模 → 计算 → 可视化 → 分析与决策

■ Matplotlib 是 Python 最基础、最通用的绘图库

数学结果 $\xrightarrow{\text{Matplotlib}}$ 可解释的信息

Matplotlib 的安装与基本定位

- **Matplotlib 是 Python 科研绘图的基础设施**
 - NumPy: 数值计算
 - Matplotlib: 二维/三维绘图底座
 - SciPy / Pandas / Seaborn / Plotly: 基于 Matplotlib 的高层封装

1. 画出第一张“有信息的图”
2. 多条曲线、图例
3. 图形定制：绘图属性

Matplotlib 的安装与基本定位

■ 推荐安装方式（科研环境）

```
# VS code 下的安装，终端：  
pip install matplotlib
```

Matplotlib 的设计哲学

“Everything is a Figure, Axes, and Artists”

Matplotlib 的最小使用模型

■ Matplotlib 的最简绘图逻辑：

创建数据 → 描点/连线 → 修饰，渲染 → 显示

■ **pyplot** 模式（教学与快速原型）：

- 类似 MATLAB
- 隐式创建 Figure 与 Axes

■ 科研绘图最终应过渡到面向对象接口

1. 画出第一张“有信息的图”
2. 多条曲线、图例
3. 图形定制：绘图属性

第一张 Matplotlib 图（离散点）

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]
plt.plot(x, y)
plt.show()
```

此时 Matplotlib 在做什么？

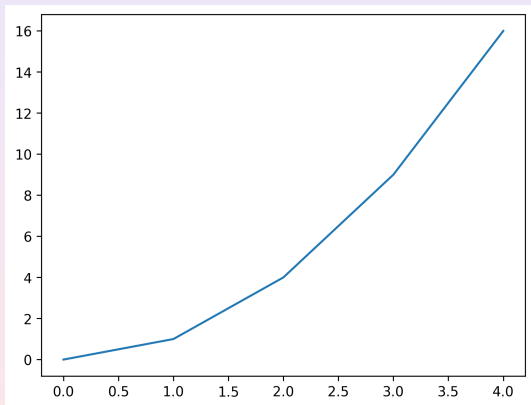
- 接收 x, y 两个 **有限离散点集**
- 默认使用 **直线插值** 连接相邻点
- 本质仍是：**描点 + 连线**

1. 画出第一张“有信息的图”

2. 多条曲线、图例

3. 图形定制：绘图属性

第一张 Matplotlib 图



使用 NumPy “加密” 数据点

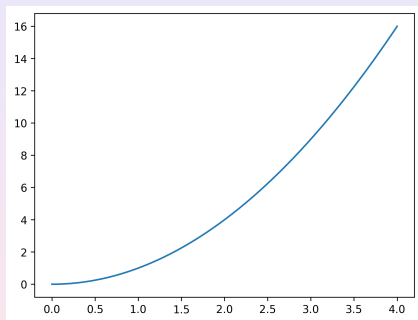
- 上一页图像 并非函数连续图像
- 我们通过加密采样点 来逼近连续函数

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 4, 200)    # 加密采样
y = x**2
plt.plot(x, y)
plt.show()
```

1. 画出第一张“有信息的图”
2. 多条曲线、图例
3. 图形定制：绘图属性

使用 NumPy “加密” 数据点



关键变化

离散 5 点 → 连续视觉效果 (200 点)

“函数曲线”并不存在

Matplotlib 的绘图本质

所有图像，都是有限点的可视化结果

- 计算机中：
 - 没有真正的连续函数
 - 只有 采样点 + 插值规则
- “曲线光滑”的来源
 - 采样点足够密
 - 人眼分辨率有限

描点法 (Sampling & Rendering)

核心思想

绘图 = 对函数进行采样 + 对采样点进行渲染

■ 数学对象:

$$y = f(x)$$

■ 数值世界:

$$\{(x_i, f(x_i))\}_{i=1}^N$$

■ 可视化世界:

scatter (点) / plot (连线)

这一认知非常重要

- 理解描点法后，你将自然理解：
 - 数值误差对图形的影响
 - 步长选择（`linspace`）
 - 插值、平滑、滤波
 - 科研绘图中“假平滑”的风险
- Matplotlib 不是画函数，而是画数据

一张合格图像的基本要素

科研绘图的最低合格标准

- 横轴含义 (**x-label**)
- 纵轴含义 (**y-label**)
- 图像标题 (**title**)
- 必要时添加图例 (**legend**)

没有标签的图 = 不完整的数学表达

为什么没有标签是严重问题？

■ 数学中，一个表达式必须包含：

自变量 + 因变量 + 含义

■ 没有标签的图像：

- 不知道变量物理意义
- 不知道单位
- 不知道适用范围

■ 在论文评审中：

- = 信息不完整
- = 专业性不足

1. 画出第一张“有信息的图”
2. 多条曲线、图例
3. 图形定制：绘图属性

加入标签后的最小合格示例

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 4, 200)
y = x**2
plt.plot(x, y, label=r'$y=x^2$')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Quadratic Function')
plt.legend()
plt.show()
```

这张图现在包含了什么信息？

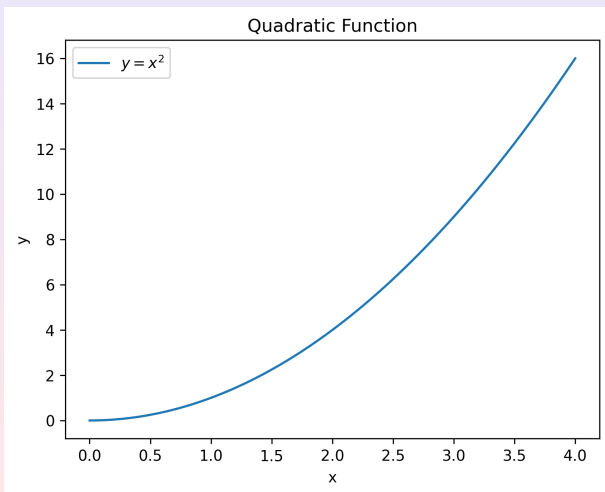
- 数学关系： $y = x^2$ ；自变量 / 因变量；函数类型与研究对象

1. 画出第一张“有信息的图”

2. 多条曲线、图例

3. 图形定制：绘图属性

升级版：带标签的折线图



标题的真正作用

标题回答的问题

“这张图想说明什么?”

■ 好标题应包含:

- 研究对象
- 关系或结论

■ 错误示例:

- Figure 1
- Result

■ 推荐风格:

- Relationship between x and x^2
- Quadratic Growth with Respect to x

图例（Legend）什么时候必须出现？

判别标准

图中是否存在多个“信息对象”？

■ 必须使用图例的情况：

- 多条曲线
- 多组实验结果
- 理论值 vs 数值解

■ 不建议省略图例：

- 用颜色“猜”
- 在正文中反复解释

1. 画出第一张“有信息的图”
2. 多条曲线、图例
3. 图形定制：绘图属性

科研绘图的第一条铁律

黄金标准

一张图必须能脱离代码和正文被理解

- 审稿人可能只看图
- PPT 听众只看到图
- 多年后的你只记得图

从单条曲线到多条曲线

■ 实际科研问题中，几乎不可避免地出现：

- 不同参数下的结果
- 理论解 vs 数值解
- 算法 A / B / C 对比

■ 一张图中出现 **多个信息对象**

⇒ 图像复杂度显著上升

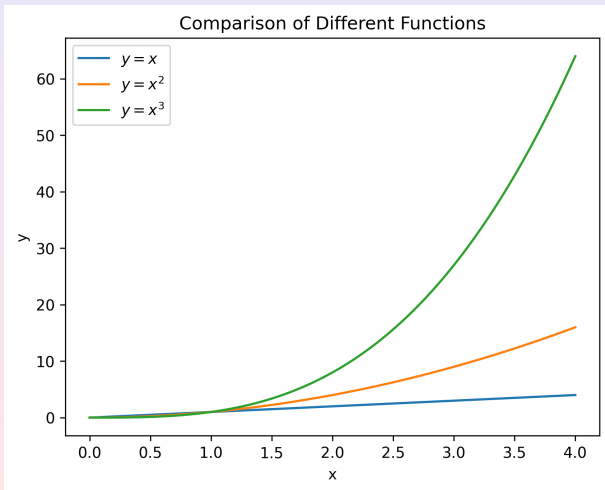
多条曲线的最小绘图示例

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 4, 200)
plt.plot(x, x, label=r'$y=x$')
plt.plot(x, x**2, label=r'$y=x^2$')
plt.plot(x, x**3, label=r'$y=x^3$')

plt.xlabel('x')
plt.ylabel('y')
plt.title('Comparison of Different Functions')
plt.legend()
plt.show()
```

多条曲线的最小绘图 legend 示例



legend 函数的基本语法

最常用调用方式

```
plt.legend()
```

- 图例内容来自每条曲线的 label 参数
- 未设置 label 的对象不会出现在图例中

legend 函数的基本语法

常用参数

```
plt.legend(  
    loc='best',  
    fontsize=12,  
    frameon=True  
)
```

- loc: 图例位置（自动 / 指定）
- fontsize: 图例字体大小
- frameon: 是否显示边框

legend 的语义原则

图例不是说明文字

legend 的职责是“标识”，不是“解释”

■ 推荐：

- 简洁数学符号： $y = x^2$
- 清晰实验标识：Method A / Method B

■ 不推荐：

- 过长自然语言
- 与标题重复的信息

为什么多条曲线必须使用图例？

核心判断

当图中存在多个语义对象时，必须显式标注

- 没有图例会导致：
 - 读者只能“猜”
 - 颜色或线型无法形成稳定语义
- 图例的本质是：

视觉对象 $\longleftrightarrow$ 数学 / 物理含义

本讲的主要内容

1 1. 画出第一张“有信息的图”

2 2. 多条曲线、图例

3 3. 图形定制：绘图属性

■ 3.1 绘图属性概述

■ 3.2 字体大小、线型、颜色与点标记

■ 3.3 图形显示中文

Line2D 对象与绘图属性总览

Matplotlib 中的一条“线”是什么？

`plt.plot()` 实际创建的是 Line2D 对象

- 所有线图的视觉表现，均由 **Line2D** 属性控制
- 绘图用到的参数，本质上都是在设置 Line2D

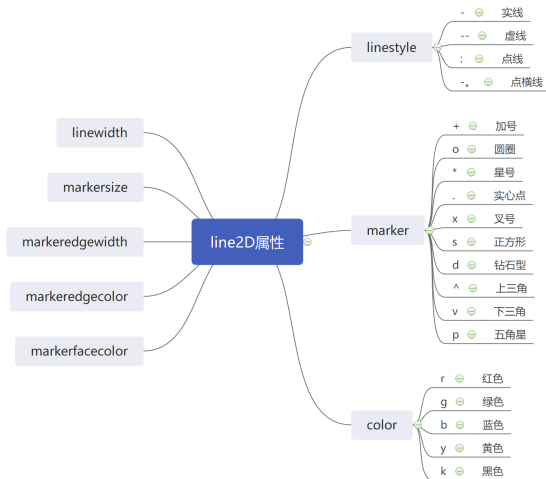
一条线 = 数据 + 一组 Line2D 属性

Line2D 对象与绘图属性总览

常用 Line2D 绘图属性

属性名	作用
marker	点的形状
markersize	点的大小
linestyle	线型（实线 / 虚线等）
linewidth (lw)	线宽
color	颜色
label	图例文本

Line2D 对象与绘图属性总览



本讲的主要内容

1 1. 画出第一张“有信息的图”

2 2. 多条曲线、图例

3 3. 图形定制：绘图属性

■ 3.1 绘图属性概述

■ 3.2 字体大小、线型、颜色与点标记

■ 3.3 图形显示中文

字体大小（fontsize）的基本语法

最常见使用位置

```
plt.xlabel('x', fontsize=14)
plt.ylabel('y', fontsize=14)
plt.title('Comparison of Functions', fontsize
          =16)
plt.legend(fontsize=12)
```

- `fontsize` 是显式控制可读性的核心参数
- 不依赖默认值，是科研绘图的基本要求

字体大小（fontsize）的基本语法

坐标刻度字体（tick labels）

```
plt.xticks(fontsize=12)
```

```
plt.yticks(fontsize=12)
```

- 刻度是**定量信息**，不可忽视

fontsize 的科研经验法则

经验推荐（非硬性规定）

元素	推荐字号
标题（Title）	16–18
坐标轴标签	14–16
刻度（Ticks）	12–14
图例（Legend）	12–14

- 投影展示 $\Rightarrow$ 字体应更大
- 论文排版 $\Rightarrow$ 缩印后仍需可读

线型、颜色与点标记的基本语法

plot 函数的常用视觉参数

```
plt.plot(  
    x, y,  
    color='blue',  
    linestyle='--',  
    linewidth=2,  
    marker='o',  
    markersize=6,  
    label='Example'  
)
```

线型、颜色与点标记的基本语法

- **color**: 颜色（区分对象）
- **linestyle**: 线型（编码语义）
- **linewidth / lw**: 线宽（强调程度）
- **marker**: 点标记（离散性）

Markers: 强调“点”的存在

常见 marker 类型

标记	符号	标记	符号
圆点	'o'	方块	's'
三角	'^'	菱形	'D'
叉号	'x'	加号	'+'

■ marker 用于强调:

- 离散采样点
- 实验 / 仿真结果

线宽（linewidth / lw）

线宽的作用

线宽是“重要性”的视觉表达

■ 常用写法：

- linewidth=2

- lw=2

■ 使用建议：

- 主结果：较粗

- 对比结果：较细

颜色的指定方式

三种常见颜色指定方式

- 颜色名称: **'red', 'blue', 'black'**
- 简写代码: **'r', 'b', 'k'**
- RGB / HEX: **'#1f77b4'**
- 科研推荐:
 - 高对比
 - 可黑白打印

常见线型（Line styles）

常用线型

线型	参数
实线	'-'
虚线	'- -'
点线	':'
点划线	'-.'

- 线型是颜色的补充编码
- 黑白打印时尤为重要

综合示例：多视觉编码联合使用

```
x = np.linspace(0, 4, 20)
plt.plot(x, x,
         color='black', linestyle='-',
         linewidth=2, marker='o',
         label=r'$y=x$')
plt.plot(x, x**2,
         color='blue', linestyle='—',
         linewidth=2, marker='s',
         label=r'$y=x^2$')
plt.plot(x, x**3,
         color='red', linestyle='-.',
         linewidth=2, marker='^',
         label=r'$y=x^3$')
plt.legend(fontsize=12)
plt.show()
```

科研绘图的视觉编码原则

- 不要只依赖颜色
- 线型、marker、线宽应 **协同编码**
- 同一语义对象风格保持一致
- 视觉差异必须对应语义差异

一句话总结

所有“好看的差异”，都应该是“有意义的差异”

网格（grid）示例：定量读取更清晰

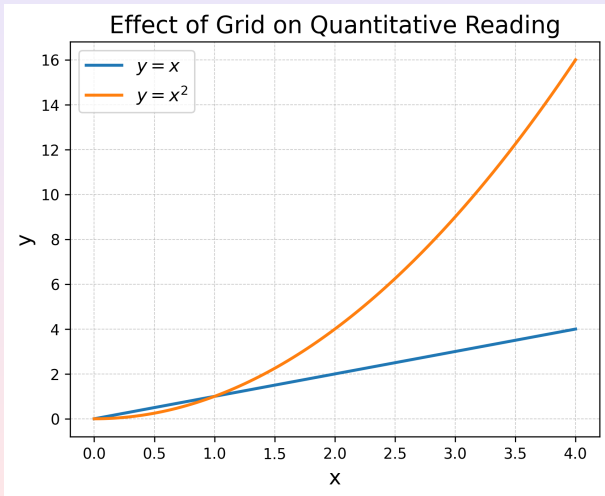
```
x = np.linspace(0, 4, 200)
y1 = x
y2 = x**2

plt.plot(x, y1, label=r'$y=x$', linewidth=2)
plt.plot(x, y2, label=r'$y=x^2$', linewidth=2)
plt.xlabel('x', fontsize=14)
plt.ylabel('y', fontsize=14)
plt.title('Effect of Grid on Quantitative Reading', fontsize=16)
plt.grid(True, linestyle='—', linewidth=0.5, alpha=0.7)
plt.legend(fontsize=12)
plt.show()
```

观察要点

- 网格线与刻度对齐，便于估计数值

网格（grid）示例：定量读取更清晰



网格 (grid): 定量读取的关键

grid 的作用

- 辅助数值读取
- 增强不同曲线的对比
- 提升图像的定量感与专业度

最基本用法

```
plt.grid(True)
```

- 默认对 x / y 轴同时生效
- 使用当前坐标轴的刻度位置

grid 的参数与使用原则

常用参数

```
plt.grid(  
    True,  
    linestyle='--',  
    linewidth=0.5,  
    alpha=0.7  
)
```

- linestyle: 网格线型（建议虚线）
- linewidth: 线宽（应弱于主曲线）
- alpha: 透明度（避免喧宾夺主）

本讲的主要内容

1 1. 画出第一张“有信息的图”

2 2. 多条曲线、图例

3 3. 图形定制：绘图属性

■ 3.1 绘图属性概述

■ 3.2 字体大小、线型、颜色与点标记

■ 3.3 图形显示中文

中文绘图显示问题

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2, 2, 200)
y = x**2
plt.plot(x, y, linewidth=2)
plt.xlabel("自变量 x", fontsize=14)
plt.ylabel("函数值 y", fontsize=14)
plt.title("二次函数示意图", fontsize=16)
plt.grid(True)
plt.show()
```

1. 画出第一张“有信息的图”

2. 多条曲线、图例

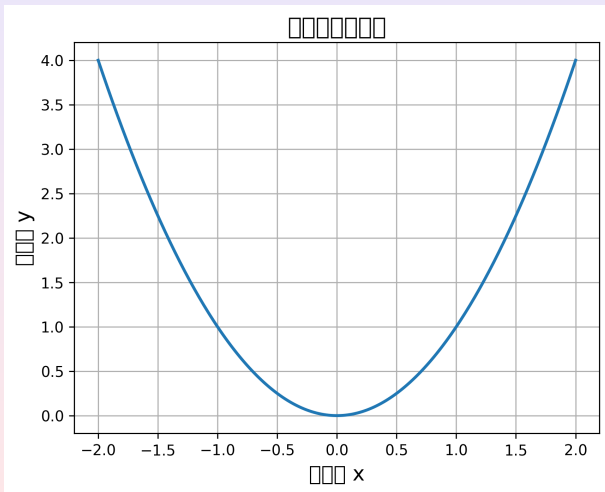
3. 图形定制：绘图属性

3.1 绘图属性概述

3.2 字体大小、线型、颜色与点标记

3.3 图形显示中文

中文绘图显示问题



Matplotlib 中的中文与负号问题

常见现象

- 中文标题 / 标签显示为方块
- 坐标轴负号 “-” 显示为乱码或缺失
- Matplotlib 默认字体：
 - 不包含中文字符
 - 使用 Unicode 负号 (U+2212)
- 结果：

字体不支持 $\Rightarrow$ 无法正确渲染

设置中文字体与负号显示

设置中文字体

```
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"] = ["SimHei"]
# 设置中文字体
```

- font.sans-serif: 无衬线字体列表
- SimHei: 黑体，常见中文字体

解决负号乱码问题

```
plt.rcParams["axes.unicode_minus"] = False
```

- 使用 ASCII - 替代 Unicode 负号

中文绘图完整示例

```
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

x = np.linspace(-2, 2, 200)
y = x**2
plt.plot(x, y, linewidth=2)
plt.xlabel("自变量 x", fontsize=14)
plt.ylabel("函数值 y", fontsize=14)
plt.title("二次函数示意图", fontsize=16)
plt.grid(True)
plt.show()
```

1. 画出第一张“有信息的图”

2. 多条曲线、图例

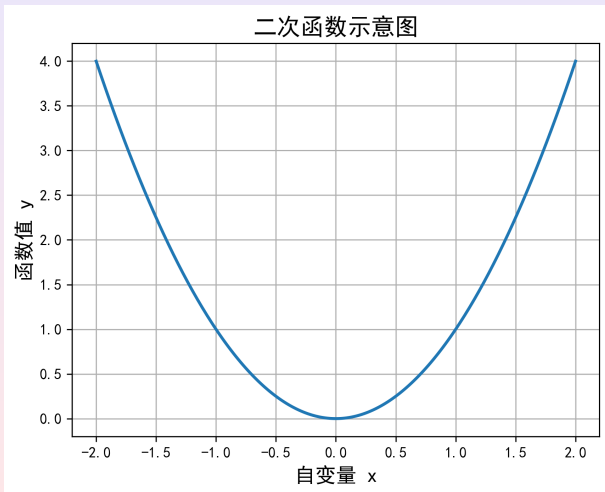
3. 图形定制：绘图属性

3.1 绘图属性概述

3.2 字体大小、线型、颜色与点标记

3.3 图形显示中文

中文绘图完整示例



1. 画出第一张“有信息的图”

2. 多条曲线、图例

3. 图形定制：绘图属性

3.1 绘图属性概述

3.2 字体大小、线型、颜色与点标记

3.3 图形显示中文

Thanks!